

What a junction is: myosin through a cell division

prototype/ecm/test_01b_myosin_pools.py · log/okuda_ECM/01b_myosin_pools · 10 August 2026

1. The question, which is about geometry and not about myosin

A cell–cell junction is a *surface*: the lateral interface between two cells, of order $\ell \times h$ where ℓ is its apical extent and h the cell’s height. The vertex model does not have h — the epithelium is one triangulated shell, a cell is a face, and a cell’s “volume” is the wedge from the tissue centre — so the junction enters the energy as a *line* of length ℓ_e , with everything about the third dimension absorbed into the line tension Λ . That is not a compromise but the right geometry for the term it multiplies: the contractile actomyosin of an adherens junction is a *belt* at the apical rim of that interface, a cable of length ℓ_e , whereas what is spread over the lateral *area* is cadherin adhesion, which this energy does not carry as a separate term.

The consequence is a statement about what the symbol m_e in $\Lambda \sum_e m_e \ell_e$ means. It is a tension multiplied by a length, so m_e is a **density** — myosin per unit length of belt — and the *amount* on a junction is $N_e = m_e \ell_e$. Everything below follows from that one identification, and the reason it is worth stating is that the model’s own inheritance rule is only correct if it is true.

2. A division does nothing to a junction, and the model must agree

When `divide_3d` inserts a vertex n into an existing contact (a, b) , it cuts that contact into (a, n) and (n, b) . *Nothing physical has happened*: the same two cells still touch across the same interface, and the only thing that changed is the mesh’s description of it. Two separate things must therefore survive the cut, and the archived model gets one of them and not the other.

The amount, which it gets right. `junction_ops._lookup` copies the parent’s m onto both halves. That is conservative *because m is a density*: $m(\ell_{an} + \ell_{nb}) = m \ell_{ab}$. Had m been an amount, the identical line of code would have doubled the tissue’s myosin at every division, and nothing would have failed — the arrays are the same shape and the run completes. The same trap sits one level up, on the faces: `divide_3d`’s `keep` map copies each per-face array onto both daughters, so anything carried that way must likewise be intensive.

The setpoint, which it gets wrong. The value each half then relaxes toward is $m_e^{ss} = a \ell_e / \langle \ell \rangle$, and ℓ_e is *extensive*: the cut halves it. So each half, having correctly inherited the parent’s myosin, immediately begins decaying toward half of it, over τ , for a reason that is a re-meshing artifact. The rule being violated is general:

the drive of a feedback must be intensive.

Tension is (cutting a cable in two does not halve the tension in it); strain and strain rate are; length is not. This is a stronger argument for keying on tension than the sign argument of `spheroid_ecm.tex` §6, because it is about the law being well posed under re-meshing rather than about which measurement to believe.

3. Two pools, one conserved amount

The repair is not a better drive but a conserved *amount* with a *derived* density — and the biology hands over the structure for free. An epithelial cell has two apical actomyosin populations, and germband extension uses both (Munjal et al., Nature 524:351, 2015): a **medioapical** meshwork spread over the apical face, and the **junctional** belt at the rim, coupled by outward flow. The first is areal, the second linear. So the answer to “should myosin scale with area or with length?” is *both* — they are two pools:

$$M_f = \rho_f A_f, \quad \frac{dM_f}{dt} = k_{\text{on}} A_f - \frac{M_f}{\tau_{\text{med}}} - \sum_{e \in \partial f} J_{f \rightarrow e}, \quad J_{f \rightarrow e} = k_{\text{ex}} \rho_f \ell_e (1 + \beta_T (T_e / \langle T \rangle - 1)), \quad (1)$$

$$N_e = n_e \ell_e, \quad \frac{dN_e}{dt} = \sum_{f \ni e} J_{f \rightarrow e} - \frac{N_e}{\tau_{\text{jun}}}, \quad m_e = a n_e / \langle n \rangle. \quad (2)$$

Every rate is per what. k_{on} is myosin assembled per unit *apical area* per frame, which is what makes the cortical pool areal rather than a per-cell budget; k_{ex} is what each unit *length* of belt draws from the cortex per frame, so the export is $\propto \ell_e$ because there is ℓ_e of belt to receive it; τ_{med} and τ_{jun} are turnover times in frames.

What is stored is the density in both cases, and that is what makes the two existing inheritance rules — copy onto both halves, copy onto both daughters — conserve the amount rather than double it. **And the**

setpoint is now intensive: the belt's supply is per unit length, so $n_e^* = \tau_{\text{jun}} k_{\text{ex}} \rho_f$ carries no ℓ_e , and half a junction relaxes toward what the whole junction was relaxing toward.

One prediction arrives unbidden. The steady state of Eq. (1) is $\rho^* = k_{\text{on}} / (1/\tau_{\text{med}} + k_{\text{ex}} P_f / A_f)$: a cell with more perimeter per unit area drains faster and holds less medioapical myosin. P_f / A_f is fixed by shape alone, so the model says — with nothing added to make it say so — that as a tissue divides into smaller cells, myosin shifts off the cortex and onto the belts.

4. As a Plexus2 container

object	kind	acts on	carries
medioapical_myosin	Lateral	cell	areal density ρ_f ; emits the flux $J_{f \rightarrow e}$
junction_myosin_model_two_pool	Structural	junction (edge set on vertex)	line density n_e ; integrates N_e
junction_myosin_sync	Rewire	junction	re-keys m_e , N_e after the topology operators

Three placements carry the argument. `two_pool` is registered as a `model=` of the existing contract and not an `implementation=`: its parameters $\{\tau_{\text{jun}}, a\}$ are disjoint from the default's $\{a, \tau, \beta, \text{keyed_on}\}$, so there is no common operating point at which to call them two ways of computing one thing — per `plexus2`, swapping a model *is* an experiment. They share the contract's kind and write the same channel `m["myo"]`, so `shape_energy_3d` cannot tell them apart and the comparison is fair. And the medioapical operator is scheduled *before* the belt's, so a frame's flux is the flux that frame's cells produced.

The sync operator is why any of this is measurable. `junction_myosin` writes `m["myo"]` sized to the half-edge buffer at its own slot; `reconnect_t1_3d` rewires those arrays and `divide_3d` lengthens them later in the same frame, and `topo_snapshot_3d` then recorded the frame's edges beside the previous topology's myosin. Measured on the 401-frame nominal: 56 of 200 snapshots carried a myosin array 6 to 1,356 entries short of the edge arrays, the two half-edges of one junction disagreed by 0.216 on those and by 0.0011 on the rest, and a tracked junction moved 0.1415 in myosin across a misaligned frame against 0.0095 across a clean one — with the leaky integrator at $\tau = 20$ unable to move more than 0.03. Every division frame, which is precisely where this note's measurement lives, was one of the broken ones. The repair keys the state by vertex pair and re-applies it after the topology operators, and is bit-identical on every position, division and T1 to the run without it.

5. What it measures

Two 401-frame builds of the same tissue differing only in the myosin model, and within each, junctions a division *cut* against *intact* ones over the same interval. The control is not optional: two frames of ordinary dynamics move an untouched junction too, and without it every division looks like a leak.

	one-pool	two-pool	reading
cuts detected	13,083	12,320	
$(N_1 + N_2) / N_{\text{parent, cut}}$	1.0270	1.0222	
$(\ell_1 + \ell_2) / \ell_{\text{parent}}$	1.0201	1.0178	the halves are 2% longer: the path bends at n
ratio of the two	1.0068	1.0043	<i>the amount is conserved</i> , to the geometry
same ratio, intact	1.0092	1.0093	and that is what two frames of dynamics do anyway
$m_{\text{half}}^{ss} / m_{\text{parent}}^{ss}$	0.514	—	<i>the setpoint halves</i> , as predicted
m after 20 frames, cut	0.815	0.956	both relative to the tissue mean
same, intact	1.023	1.019	
cut / intact	0.797	0.939	a 20% loss to re-meshing, against 6%
T1 per cell per frame	0.00221	0.00373	1.69×; neighbour exchange is the observable

The residual 6% is not a leftover artifact. Writing $N_e = n_e \ell_e$ in Eq. (2) gives $\dot{n}_e = k_{\text{ex}} \rho - n_e / \tau_{\text{jun}} - n_e d \ln \ell_e / dt$: a junction whose length is *changing* dilutes or concentrates its own belt. Freshly cut halves relax and lengthen, so they dilute. That term is advection of a conserved species along a moving cable, which is physics; the 20% it replaces was arithmetic.

6. What this does and does not license

It licenses the reading of m_e as a density and therefore the inheritance rules that were already written; it licenses the two-pool structure as the smallest thing that makes the junctional setpoint survive re-meshing, at the cost of

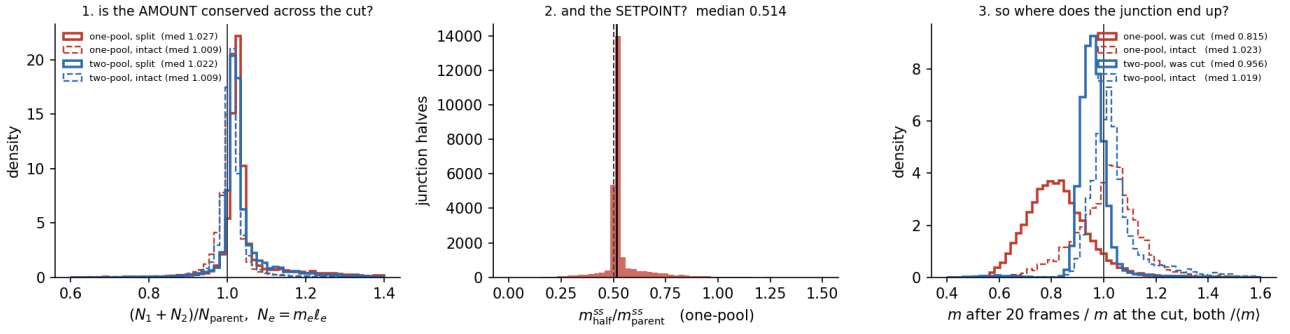


Figure 1: Left: the amount across a cut, both models, against the intact control — the offset from 1 is the 2% by which the two halves are longer than the parent they replace, not a leak. Middle: the one-pool setpoint of a half over its parent's, which is $1/2$ because the setpoint is proportional to a length that was just halved. Right: where the junction actually ends up twenty frames later. The one-pool cut junctions sit 20% below their intact neighbours for no physical reason; the two-pool ones sit 6% below.

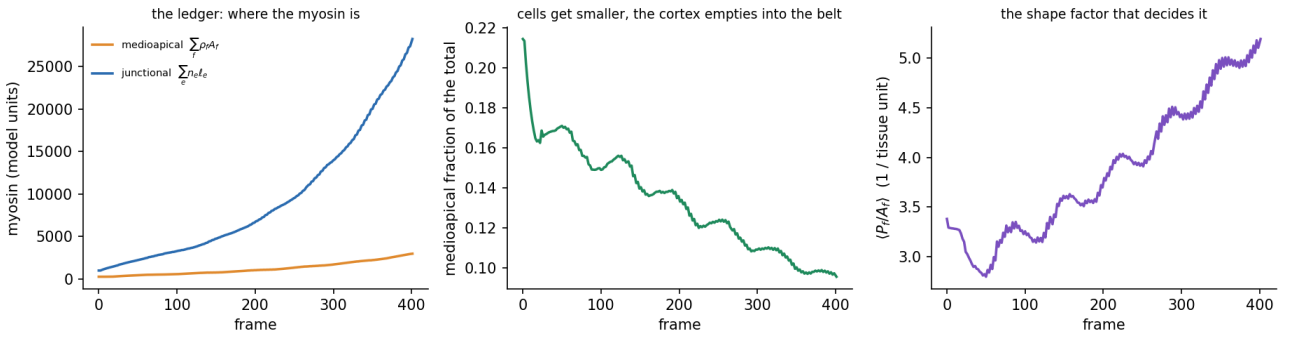


Figure 2: The two-pool ledger. Left: both pools grow with the tissue. Middle: the medioapical share falls from 0.214 to 0.096. Right: why — the mean shape factor $\langle P_f/A_f \rangle$ rises from 3.38 to 5.19 as cells get smaller, and the steady state of Eq. (1) depends on it. Nothing in the specification says myosin should move from the cortex to the belt; the geometry does.

one new operator and four rate constants; and it licenses the claim that the model's own bookkeeping, not its biology, was responsible for a fifth of the myosin on every junction a division touched.

It does not license the operating point. k_{on} was set to $1/\tau_{\text{med}} + k_{\text{ex}}\langle P/A \rangle_0 = 0.219$ so the cortical density *starts* at its own steady state rather than relaxing to it over the first fifty frames, and $\tau_{\text{med}} = \tau_{\text{jun}} = 20$ frames and $k_{\text{ex}} = 0.05$ are chosen numbers, not measured ones — and one frame is still not anchored to a time, so “20 frames” is not yet a claim about myosin turnover, which is 1–2 minutes. It does not license β_T : the run reported here has $\beta_T = 0$, pure transport, deliberately, so that a tension bias added later has to beat what geometry already does. And it does not contain pulsatility, which is the mechanism the medioapical pool is famous for: Munjal's pulses need advection-mediated positive feedback with dissociation as the negative arm, which is a third state variable and an advection term. That is the next operator, and this one is the control it has to beat.